Systemd Timers for Scheduling Tasks

Posted by Richard England on June 18, 2021

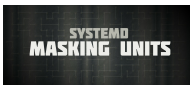
IN THIS SERIES →

1. What is an init system?

2. systemd unit file basics

3. systemd: Converting sysvinit scripts

4. systemd: Using the journal



5. systemd: Masking units

6. systemd: Unit dependencies and order

7. systemd: Template unit files

8. Systemd Timers for Scheduling Tasks

Systemd has provided timers for a while and it is worth taking a look at this feature as an alternative to *cron*. This article will show you how to use timers with *systemd* to run a task after the system has booted and on a recurring basis after that. This is not a comprehensive discussion of *systemd*, only an introduction to this one feature.

Cron vs anacron vs systemd: a quick review

Cron can schedule a task to be run at a granularity ranging from minutes to months or more. It is relatively simple to set up, requiring a single configuration file. Although the configuration line is somewhat esoteric. It can also be used by general users.

Cron, however, fails if your system happens to not be running when the appropriate execution time occurs.

Anacron, overcomes the "system not running" issue. It insures that the task will be executed when your system is again active. While it was intended to be used by administrators, some systems give general users access.

However, the *anacron* frequency of execution can be no less than daily.

Both *cron* and *anacron* have issues with consistency in execution context. Care must be taken that the environment in effect when the task runs is exactly that used when testing. The same shell, environment variables and paths must be provided. This means that testing and debugging can sometimes be difficult.

Systemd timers offer the best of both *cron* and *anacron*. Allows scheduling down to minute granularity. Assures that the task will be executed when the system is again running even if it was off during the expected execution time. Is available to all users. You can test and debug the execution in the environment it will run in.

However, the configuration is more involved, requiring at least two configuration files.

If your *cron* and *anacron* configuration is serving you well then there may not be a reason to change. But *systemd* is at least worth investigating since it may simplify any current *cron* /*anacron* work-arounds.

Configuration

Systemd timer executed functions require, at a minimum, two files. These are the "timer unit" and the "service unit". Actions consisting of more than a simple command, you will also need a "job" file or script to perform the necessary functions.

The timer unit file defines the schedule while the service unit file defines the task(s) performed. More details on the .timer unit is available in “man systemd.timer”. Details on the service unit are available in “man systemd.service”.

There are several locations where unit files exist (listed in the man page). Perhaps the easiest location for the general user, however, is “~/.config/systemd/user”. Note that “user” here, is the literal string “user”.

Demo

This demo is a simple example creating a user scheduled job rather than a system schedule job (which would run as root). It prints a message, date, and time to a file.

1. Start by creating a shell script that will perform the task. Create this in your local “bin” directory, for example, in “~/bin/schedule-test.sh”

To create the file:

```
touch  ~/bin/schedule-test.sh
```

Then add the following content to the file you just created.

```
#!/bin/sh
echo "This is only a test: $(date)" >> "$HOME/schedule-test-output.txt"
```

Remember to make your shell script executable.

2. Create the .service unit that will call the script above. Create the directory and file in: “~/.config/systemd/user/schedule-test.service”:

```
[Unit]
Description=A job to test the systemd scheduler

[Service]
Type=simple
ExecStart=/home/<user>/bin/schedule-test.sh

[Install]
WantedBy=default.target
```

Note that <user> should be your @HOME address but the “user” in the path name for the unit file is literally the string “user”.

The *ExecStart* line should provide an absolute address with no variables. An exception to this is that for *user* units you may substitute “%h” for $HOME. In other words you can use:

```
ExecStart=%h/bin/schedule-test.sh
```

This is only for user unit file use. It is not good for system units since “%h” will always return “/root” when run in the system environment. Other substitutions are found in “man systemd.unit” under the heading “SPECIFIERS”. As it is outside the scope of this article, that’s all we need to know about SPECIFIERS for now.

3. Create a .timer unit file which actually schedules the .service unit you just created. Create it in the same location as the .service file “~/.config/systemd/user/schedule-test.timer”. Note that the file names differ only in their extensions, that is ,”.service” versus “.timer”

```
[Unit]
Description=Schedule a message every 1 minute
RefuseManualStart=no  # Allow manual starts
RefuseManualStop=no   # Allow manual stops

[Timer]
#Execute job if it missed a run due to machine being off
Persistent=true
#Run 120 seconds after boot for the first time
OnBootSec=120
```

```
#Run every 1 minute thereafter
OnUnitActiveSec=60
#File describing job to execute
Unit=schedule-test.service

[Install]
WantedBy=timers.target
```

Note that the .timer file has used “OnUnitActiveSec” to specify the schedule. Much more flexible is the “OnCalendar” option. For example:

```
# run on the minute of every minute every hour of every day
 OnCalendar=*-*-* *:*:00
# run on the hour of every hour of every day
 OnCalendar=*-*-* *:00:00
# run every day
 OnCalendar=*-*-* 00:00:00
# run 11:12:13 of the first or fifth day of any month of the year
# 2012, but only if that day is a Thursday or Friday
 OnCalendar=Thu,Fri 2012-*-1,5 11:12:13
```

More information on “OnCalendar” is available here.

4. All the pieces are in place but you should test to make certain everything works. First, enable the user service:

```
$ systemctl enable schedule-test.service
```

This should result in output similar to this:

```
Created symlink
/home/<user>/.config/systemd/user/default.target.wants/schedule-
test.service → /home/<user>/.config/systemd/user/schedule-test.service.
```

Now do a test run of the job:

```
$ systemctl start schedule-test.service
```

Check your output file ($HOME/schedule-test-output.txt) to insure that your script is
performing correctly. There should be a single entry since we have not started the timer yet. Debug as necessary. Don’t forget to enable the service again if you needed to change your .service file as opposed to the shell script it invokes.

5. Once the job works correctly, schedule it for real by enabling and starting the user timer for your service:

```
$ systemctl enable schedule-test.timer
$ systemctl start schedule-test.timer
```

Note that you have already started and enabled the service in step 4, above, so it is only necessary to enable and start the timer for it.

The “enable” command will result in output similar to this:

```
Created symlink
/home/<user>/.config/systemd/user/timers.target.wants/schedule-
test.timer → /home/<user>/.config/systemd/user/schedule-test.timer.
```

and the “start” will simply return you to a CLI prompt.

Other operations

You can check and monitor the service. The first command below is particularly useful if you receive an error from the service unit:

```
$ systemctl status schedule-test
$ systemctl list-unit-files
```

Manually stop the service:

```
$ systemctl stop schedule-test.service
```

Permanently stop and disable the timer and the service, reload the daemon config and reset any failure notifications:

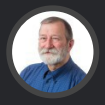
```
$ systemctl stop schedule-test.timer
$ systemctl disable schedule-test.timer
$ systemctl stop schedule-test.service
$ systemctl disable schedule-test.service
$ systemctl daemon-reload
$ systemctl reset-failed
```

Summary

This article will jump-start you with *systemd* timers, however, there is much more to *systemd* than covered here. This article should provide you with a foundation on which to build. You can explore more about it starting in the Fedora Magazine systemd series .

References – Further reading:

- man systemd.timer
- man systemd.service
- Use systemd timers instead of cronjobs
- Understanding and administering systemd
- Also, https://opensource.com/ has a good systemd cheatsheet



Richard England

Unix/Linux enthusiastic user for 40 years. Customer software support engineer for most of that time specializing in creating/improving documentation using customer input from support issues. Now retired.